Preventive Maintenance Web App — Business Requirements Document

Version: 2.0
Date: 2026-08-22
Prepared for: YDC IT leadership and process stakeholders
Companion documents: `PRD.md` (what the system does, in functional detail), `SOLUTION.md` (how it is built), `USER_GUIDE.md` (how to use it)

1. Executive summary

YDC's IT department is accountable for preventive maintenance (PMS) on every in-production Service Desk and Infrastructure & Security asset, three times a year. Before this system, that accountability was tracked by hand in a shared spreadsheet: a technician (or, in practice, whoever remembered to) ticked a checkbox and typed a remark directly into the same workbook the asset inventory lived in, with no enforced process, no audit trail of who did what, no way to tell a genuinely completed inspection from an untouched cell, and no way to know a finding had actually been fixed rather than just written down.

This system replaces that manual process with a governed web application: a technician is identified automatically, is limited to the assets they are actually responsible for, is walked through a structured checklist for their asset type, and cannot mark work "done" without meeting a defined bar. What used to be a spreadsheet cell is now a decision the system defends. The result is that the business can, at any moment, answer "are we compliant this cycle" with a number it can trust, "who actually did this maintenance" with a name it can trust, and "was every finding actually fixed" with a status it can trust — none of which the prior process could reliably answer.

2. Business problem

Prior to this system:

1. **No enforced accountability.** Anyone with edit access to the shared spreadsheet could mark any asset as maintained, whether or not maintenance actually happened, and there was no record of who made that change or when.
2. **No structured evidence of what was actually checked.** A ticked checkbox said nothing about *what* was inspected — a hardware check, a security check, a network check — only that the cell was ticked.
3. **No separation between "found a problem" and "fixed the problem."** A remark could describe a fault, but nothing tracked whether that fault was ever actually resolved; a genuinely broken asset could look identical, in the tracker, to a genuinely healthy one.
4. **No reliable compliance number.** The existing dashboard counted every asset regardless of whether it was still in production, so its percentage did not answer the question leadership actually

asks: "of the assets we are responsible for right now, how many got maintained this cycle?"

5. **No mechanism to stop the same mistake twice.** A double-submission, a re-opened old record, or a well-meaning edit to an already-finished inspection could all silently corrupt the record with no warning and no trace.
6. **Growing, ungoverned data with no exit ramp.** The workbook accumulates a fixed amount of new data every cycle, forever, with no deliberate, safe way to retire old years once they were no longer needed live, short of an administrator manually deleting rows and hoping nothing important was lost.
7. **Onboarding and permission changes required a spreadsheet edit.** Adding a new technician, correcting someone's section, or delegating asset-master upkeep to someone other than IT leadership all required direct, unaudited access to the underlying workbook.

None of these are technology gaps in the abstract — they are governance gaps that happened to be implemented in a spreadsheet. The business need was a system that could enforce a process, not merely display one.

3. Business objectives

1. **Establish a defensible compliance number.** At any time, be able to state exactly what percentage of currently in-production assets received preventive maintenance this cycle, based on a definition the business agrees to and the system enforces consistently.
2. **Establish individual accountability.** Every maintenance record and every change to a findings ticket is attributable to a real, authenticated person, permanently.
3. **Close the loop between a finding and a fix.** A problem discovered during maintenance must be trackable to resolution, not just written down and forgotten.
4. **Reduce the effort required to comply.** A technician should be able to complete an inspection in the field, on a phone or a desktop, without fighting the tool.
5. **Reduce administrative overhead of running the process.** Onboarding a technician, correcting a section, delegating asset-list upkeep, and retiring old data should all be things IT leadership can do without spreadsheet surgery.
6. **Protect the historical record.** Once work is done and recorded, it must not be possible — accidentally or otherwise — for anyone, including the person who did the work, to change what history says happened.
7. **Keep the workbook itself sustainable.** The system should manage its own long-term size deliberately, with backups, rather than becoming someone's problem to fix under pressure years from now.
8. **Keep the deadline visible without requiring anyone to remember to check.** As a cycle's deadline approaches, the people responsible for finishing it should be told, automatically, rather than finding out after the fact.

4. Stakeholders

Stakeholder	Interest
IT leadership / PMS administrator	Owns the compliance target; needs a trustworthy number, an audit trail, and administrative tools that don't require touching the raw spreadsheet
Service Desk technicians	Perform the majority of PMS records; need a fast, low-friction way to complete an inspection and resume unfinished work
Infrastructure & Security technicians	Perform a smaller volume of higher-stakes inspections requiring photographic/file evidence; need the same reliability plus confidence that their evidence is genuinely preserved
Asset Managers (delegated role)	Maintain the asset master (tags, status, location) without needing full administrator access
Anyone repairing a flagged finding	May not be the original technician; needs visibility into what was found and a way to track the repair to close, regardless of section
Auditors / compliance reviewers (future)	Need a trustworthy, tamper-resistant historical record of who did what maintenance, when, and what was found

5. Business scope

5.1 In scope

- End-to-end preventive maintenance recording for both IT sections, from asset selection through a completed, locked, auditable record.
- Enforced eligibility (only in-production assets), enforced completeness (a defined bar per section before "done" is accepted), and enforced identity (no anonymous or self-declared submissions).
- Tracking a maintenance finding through to an actual repair outcome, with a permanent history of who moved it and why.
- A live, section- and organization-wide compliance view, refreshed continuously rather than on a manual schedule.
- Delegated, in-app administration of the asset master and the user roster, so routine upkeep does not require spreadsheet access.
- A controlled, backed-up way to retire old years' data as the workbook grows, and a controlled way to advance the operational tracker into a new year without losing history.
- Backfilling maintenance that was genuinely performed before this system existed, so historical compliance isn't artificially zeroed out by the system's own launch date.
- Proactive reminders as a compliance deadline approaches.

5.2 Out of scope (this version)

- Replacing the process that decides which assets exist or are in production — that remains an upstream asset-management responsibility this system reads from, not owns.
- Any workflow for a device outside the two covered sections (Service Desk, Infrastructure & Security).
- Automatic escalation beyond the single daily deadline reminder — for example, individually addressed "you personally have N assets still pending" digests, or manager escalation for a technician who is behind.
- Exportable, presentation-ready compliance reports beyond what the live dashboard shows.
- Any offline mode; a working internet connection to Google's services is required to record maintenance.
- Public or external (non-YDC) access of any kind.

6. Business requirements

Each business requirement below maps to detailed functional requirements in `PRD.md`, referenced in parentheses.

#	Business requirement	Why it matters	PRD reference
BR-1	A technician must be identified automatically from their YDC account; no shared logins, no self-declared identity	Accountability requires knowing who actually did the work	§6
BR-2	A technician sees and can act only on assets in their own registered section	Prevents cross-team data entry errors and confusion about ownership	§5, §6
BR-3	Only assets currently in production count toward eligibility and compliance	A retired or spare asset should never inflate or deflate the number leadership relies on	§9.2, §10.3
BR-4	A maintenance record cannot be marked complete without meeting a defined, section-specific bar	"Done" must mean something consistent, not "whatever the technician felt was enough"	§8, §9.2
BR-5	A finding that requires follow-up must be tracked to an actual resolution, separately from the original inspection, and visible to whoever ends up fixing it, regardless of team	Otherwise a documented problem is indistinguishable from a forgotten one	§9A
BR-6	Once a record is complete, it is permanent — nobody, including its author, can alter what history says happened	The compliance number and the audit trail must be trustworthy after the fact, not just at the moment of entry	§7B
BR-7	Every change to a repair-in-progress finding is permanently logged with who changed it, when, and why	Repairs are frequently handed between people; the history must survive that handoff	§9A.3
BR-8	Evidence submitted for Infrastructure & Security work (firmware/config proof) must be genuinely preserved and verifiably unaltered, not just "a file was attached at some point"	This evidence is what makes an Infra inspection defensible; a swapped or deleted file must be caught, not trusted	§11.1
BR-9	Administrators can onboard a technician, correct their section, delegate asset-list upkeep, or adjust active status without editing the underlying spreadsheet	Reduces operational overhead and the risk of a manual edit going wrong	§11.3, §11.4
BR-10	Administrator (top-level) access can never be granted through any in-app screen, only through direct configuration	The most sensitive permission in the system must have a deliberately higher bar than every other one, immune to a UI mistake	§5.4
BR-11	The organization can advance to a new maintenance year without losing any prior year's history, and can later retire a year it no longer needs live — with a mandatory backup first	Balances "keep everything" against "the workbook must stay usable" without ever risking silent data loss	§9.8, §9B

#	Business requirement	Why it matters	PRD reference
BR-12	Maintenance genuinely performed before this system existed can be recorded into it, so the compliance history isn't artificially incomplete	A launch date should not erase real prior work from the record	§9.6
BR-13	The people responsible for finishing a cycle's PMS are reminded automatically as the deadline nears, without depending on anyone remembering to check a dashboard	Deadlines that depend on memory get missed	§9C
BR-14	The application must work usably on both desktop and mobile devices, since maintenance is often performed at the asset, not at a desk	Field usability directly affects whether the process actually gets followed	§12

7. Success metrics

Metric	Target / direction
Live compliance percentage (completed / eligible in-production assets, per cycle)	Trending toward 100% each cycle; visible at all times, not reconstructed after the fact
Findings tickets with no status change in an unreasonable window	Trending down — a stalled repair should be visible and actionable, not invisible
Records requiring backdated/legacy import after the fact	Trending toward zero over time, as the system becomes the primary point of entry rather than a place work gets recorded after being done elsewhere
Administrator time spent on manual spreadsheet edits for roster/asset upkeep	Trending toward zero, replaced by in-app actions
Incidents of a completed record being altered after the fact	Zero, by design (§7B) — this is a hard requirement, not a trend
Time from "cycle enters its final stretch" to "every technician has been notified"	Same day, automatically, every cycle

8. Assumptions and constraints

- Every technician and administrator has a YDC Google Workspace account; there is no provision for a non-Google identity.
- The organization continues to use Google Sheets as the asset-of-record for the asset master; this system reads that master and does not replace the process that maintains it.
- Google Apps Script's platform limits (execution time per run, daily email quota, spreadsheet cell ceiling) are accepted operating constraints; the system is designed to degrade gracefully within

them (chunked processing, capacity warnings, a deliberate data-retirement tool) rather than assuming they will never be reached.
- The two Infrastructure & Security evidence folders, and the spreadsheet itself, are pre-existing Drive resources whose IDs are fixed in configuration; provisioning a new folder or workbook is outside this system's scope.
- English is the working language throughout; no localization requirement exists.

9. Key business risks and how they are addressed

Risk	Business impact if unaddressed	How the solution addresses it
A technician (accidentally or otherwise) edits a completed record	Compliance history becomes untrustworthy	Structurally impossible — enforced server-side, not just hidden in the UI (§7B)
A finding is written down but never actually fixed	An asset stays broken while the record implies it was handled	Findings tickets make "still open" visible and trackable separately from the maintenance record (§9A)
Administrator access is granted too broadly, by accident, through a convenience feature	The most sensitive permission in the system becomes easy to leak	Administrator status is deliberately excluded from every in-app write path (§5.4, BR-10)
The workbook grows without bound until it becomes slow or hits a hard platform limit	An operational failure at the worst possible time, with no plan	A dedicated, backed-up, audited Year Purge tool exists specifically so this is a deliberate decision, made early, not an emergency (§9B.2)
A cycle deadline is missed because nobody happened to check the dashboard	Compliance shortfall that was avoidable	Automatic daily reminder once the deadline is close (§9C)
Evidence for Infrastructure & Security work is later found to be swapped, altered, or missing	An inspection that looked complete turns out not to be defensible	Evidence is re-verified — not just checked once at upload — at the moment of completion (§11.1)

10. Organizational impact

- **Technicians** gain a guided, mobile-friendly workflow in place of a shared spreadsheet, at the cost of losing the ability to mark something "done" without meeting the defined bar — a deliberate trade-off in favor of a trustworthy record.
- **IT leadership / administrators** gain in-app tools for roster and asset-master upkeep and a live, trustworthy compliance view, at the cost of the discipline required to actually run the Rollover/Purge process deliberately each year rather than letting the workbook grow unmanaged.

- **Whoever repairs a flagged finding** — who may not be the technician who found it — gains visibility into open findings across both sections and a permanent record of what's been tried, rather than relying on someone remembering to mention it.
- **Training impact** is addressed by `USER_GUIDE.md`, written for the actual end users of each role (technician, Asset Manager, administrator) rather than for engineers.

11. Approval

This document, alongside `PRD.md`, represents the current, implemented, production state of the system as of the date above. Prior approval of the version 1.1 PRD (2026-08-13) authorized the original build; this revision documents functionality delivered since, for continued organizational reference rather than as a new approval gate.